**Week 9: Sales and Customer Analytics System**

The e-commerce company wants to study its sales information to understand customer buying habits, product sales performance, and overall business revenue. The management team needs useful reports to find the highest-spending customers, most popular products, and categories that generate more profit.

**1. Apply SQL Aggregate Functions**

SQL aggregate functions are used to perform calculations on a group of records and return a single result. They help summarize large amounts of data and make it easier to analyze sales and customer information.

The main aggregate functions are COUNT(), SUM(), AVG(), MIN(), and MAX(). These functions are useful for finding the total number of orders, calculating total sales, finding average order values, identifying the lowest price, and finding the highest sales amount.

**2. Generate Total Sales Reports**

1. Total number of orders placed.

```
SQL> SELECT COUNT(*) AS total_orders FROM orders;

TOTAL_ORDERS
------------
           5
```

2. Total revenue generated from sales.

```
SQL> SELECT SUM(total_amount) AS total_revenue FROM orders;

TOTAL_REVENUE
-------------
        13078
```

3. Average order value.

```
SQL> SELECT AVG(total_amount) AS average_order_value FROM orders;

AVERAGE_ORDER_VALUE
-------------------
             2615.6
```

4. Highest and lowest order amount.

```
SQL> SELECT MAX(total_amount) AS highest_order, MIN(total_amount) AS lowest_order FROM orders;

HIGHEST_ORDER LOWEST_ORDER
------------- ------------
         5071          307
```

5. Total sales for a specific period.

```
SQL> SELECT SUM(total_amount) AS period_sales FROM orders WHERE order_date >= DATE '2025-01-01' AND order_date < DATE '2025-05-01';

PERIOD_SALES
------------
        8007
```

**3. Customer Purchase Analysis**

1. Total number of orders placed by each customer.

```
SQL> SELECT c.customer_id, c.customer_name, COUNT(o.order_id) AS total_orders FROM customer c LEFT JOIN orders o ON c.customer_id = o.customer_id GR
OUP BY c.customer_id, c.customer_name;

CUSTOMER_ID CUSTOMER_NAME                                      TOTAL_ORDERS
----------- -------------------------------------------------- ------------
        103 Rahul Raj                                                 1
        102 Priya Sharma                                              1
        104 Divya Devi                                                1
        101 Arun Kumar Raj                                            1
        105 Ram                                                       1
```

2. Total amount spent by each customer.

```
SQL> SELECT c.customer_id, c.customer_name, NVL(SUM(o.total_amount), 0) AS total_spent FROM customer c LEFT JOIN orders o ON c.customer_id = o.custo
mer_id GROUP BY c.customer_id, c.customer_name;

CUSTOMER_ID CUSTOMER_NAME                                      TOTAL_SPENT
----------- -------------------------------------------------- -----------
        103 Rahul Raj                                                4800
        102 Priya Sharma                                              307
        104 Divya Devi                                                400
        101 Arun Kumar Raj                                           2500
        105 Ram                                                      5071
```

3. Average spending per customer.

```
SQL> SELECT c.customer_id, c.customer_name, NVL(AVG(o.total_amount), 0) AS average_spending FROM customer c LEFT JOIN orders o ON c.customer_id = o.
customer_id GROUP BY c.customer_id, c.customer_name;

CUSTOMER_ID CUSTOMER_NAME                                      AVERAGE_SPENDING
----------- -------------------------------------------------- ----------------
        103 Rahul Raj                                                    4800
        102 Priya Sharma                                                  307
        104 Divya Devi                                                    400
        101 Arun Kumar Raj                                               2500
        105 Ram                                                          5071
```

4. Customers with maximum purchase amount.

```
SQL> SELECT c.customer_id, c.customer_name, SUM(o.total_amount) AS total_spent FROM customer c JOIN orders o ON c.customer_id = o.customer_id GROUP
BY c.customer_id, c.customer_name HAVING SUM(o.total_amount) = (SELECT MAX(customer_total) FROM (SELECT SUM(total_amount) AS customer_total FROM ord
ers GROUP BY customer_id));

CUSTOMER_ID CUSTOMER_NAME                                      TOTAL_SPENT
----------- -------------------------------------------------- -----------
        105 Ram                                                      5071
```

5. Customers with fewer purchases.

```
SQL> SELECT c.customer_id, c.customer_name, COUNT(o.order_id) AS total_orders FROM customer c LEFT JOIN orders o ON c.customer_id = o.customer_id GR
OUP BY c.customer_id, c.customer_name HAVING COUNT(o.order_id) <= 1;

CUSTOMER_ID CUSTOMER_NAME                                      TOTAL_ORDERS
----------- -------------------------------------------------- ------------
        103 Rahul Raj                                                 1
        102 Priya Sharma                                              1
        104 Divya Devi                                                1
        101 Arun Kumar Raj                                            1
        105 Ram                                                       1
```

**4. Find Top Customers Based on Purchase Amount**

1. Top 5 customers based on total spending.

```
SQL> SELECT * FROM (SELECT c.customer_id, c.customer_name, SUM(o.total_amount) AS total_spent FROM customer c JOIN orders o ON c.customer_id = o.cus
tomer_id GROUP BY c.customer_id, c.customer_name ORDER BY total_spent DESC) WHERE ROWNUM <= 5;

CUSTOMER_ID CUSTOMER_NAME                                      TOTAL_SPENT
----------- -------------------------------------------------- -----------
        105 Ram                                                      5071
        103 Rahul Raj                                                4800
        101 Arun Kumar Raj                                           2500
        104 Divya Devi                                                400
        102 Priya Sharma                                              307
```

2. Customers with maximum number of orders.

```
SQL> SELECT c.customer_id, c.customer_name, COUNT(o.order_id) AS total_orders FROM customer c JOIN orders o ON c.customer_id = o.customer_id GROUP B
Y c.customer_id, c.customer_name HAVING COUNT(o.order_id) = (SELECT MAX(order_count) FROM (SELECT COUNT(*) AS order_count FROM orders GROUP BY custo
mer_id));

CUSTOMER_ID CUSTOMER_NAME                                      TOTAL_ORDERS
----------- -------------------------------------------------- ------------
        103 Rahul Raj                                                 1
        102 Priya Sharma                                              1
        104 Divya Devi                                                1
        101 Arun Kumar Raj                                            1
        105 Ram                                                       1
```

3. Frequent customers.

```
SQL> SELECT c.customer_id, c.customer_name, COUNT(o.order_id) AS total_orders FROM customer c JOIN orders o ON c.customer_id = o.customer_id GROUP B
Y c.customer_id, c.customer_name HAVING COUNT(o.order_id) >= 2;

no rows selected
```

4. High-value customers.

```
SQL> SELECT c.customer_id, c.customer_name, SUM(o.total_amount) AS total_spent FROM customer c JOIN orders o ON c.customer_id = o.customer_id GROUP
BY c.customer_id, c.customer_name HAVING SUM(o.total_amount) >= 50000 ORDER BY total_spent DESC;

no rows selected
```

### 5. Identify Best-Selling Products

1. Products with maximum sales quantity.

```
SQL> SELECT p.product_id, p.product_name, SUM(od.quantity) AS quantity_sold FROM product p JOIN order_details od ON p.product_id = od.product_id GRO
UP BY p.product_id, p.product_name HAVING SUM(od.quantity) = (SELECT MAX(total_quantity) FROM (SELECT SUM(quantity) AS total_quantity FROM order_det
ails GROUP BY product_id));

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
QUANTITY_SOLD
-------------
P004
Pencil Box
          20
```

2. Products generating highest revenue.

```
SQL>  SELECT p.product_id, p.product_name, SUM(od.quantity * od.price) AS revenue FROM product p JOIN order_details od ON p.product_id = od.product_
id GROUP BY p.product_id, p.product_name ORDER BY revenue DESC;

PRODUCT PRODUCT_NAME                REVENUE
------- ------------------------- ----------
P004    Pencil Box                 80000.00
P003    Pencil                     45000.00
P005    Spiral Notebook            21000.00
P002    Gel Pen                    20000.00
P001    Ball Pen                    5000.00

5 rows selected.
```

3. Least-selling products.

```
SQL> SELECT p.product_id, p.product_name, NVL(SUM(od.quantity), 0) AS quantity_sold FROM product p LEFT JOIN order_details od ON p.product_id = od.p
roduct_id GROUP BY p.product_id, p.product_name ORDER BY quantity_sold ASC;

PRODUCT PRODUCT_NAME              QUANTITY_SOLD
------- ------------------------- -------------
P016    Notebook                              0
P001    Ball Pen                              5
P002    Gel Pen                              10
P005    Spiral Notebook                      14
P003    Pencil                               15
P004    Pencil Box                           20

6 rows selected.
```

4. Products requiring promotion.

```
SQL> SELECT p.product_id, p.product_name, NVL(SUM(od.quantity), 0) AS quantity_sold FROM product p LEFT JOIN order_details od ON p.product_id = od.p
roduct_id GROUP BY p.product_id, p.product_name HAVING NVL(SUM(od.quantity), 0) <= 1;

PRODUCT PRODUCT_NAME              QUANTITY_SOLD
------- ------------------------- -------------
P016    Notebook                              0

1 row selected.
```

### 6. Perform Category-wise Sales Analysis

1. Total sales generated by each category.

```
SQL> SELECT c.category_id, c.category_name, NVL(SUM(od.quantity * od.price), 0) AS total_sales FROM category c LEFT JOIN product p ON c.category_id
= p.category_id LEFT JOIN order_details od ON p.product_id = od.product_id GROUP BY c.category_id, c.category_name;

CATEGOR CATEGORY_NAME              TOTAL_SALES
------- ------------------------- -------------
104     Art Supplies                   80000.00
105     Files and Folders              21000.00
107     Paper Products                      .00
103     Office Supplies                     .00
101     Writing Instruments            50000.00
108     Adhesives                           .00
109     Desk Accessories                    .00
106     School Supplies                     .00
102     Notebooks                      20000.00

9 rows selected.
```

2. Number of products sold in each category.

```
SQL> SELECT c.category_name, NVL(SUM(od.quantity), 0) AS products_sold FROM category c LEFT JOIN product p ON c.category_id = p.category_id LEFT JOI
N order_details od ON p.product_id = od.product_id GROUP BY c.category_name;

CATEGORY_NAME             PRODUCTS_SOLD
------------------------- -------------
Art Supplies                         20
Paper Products                        0
Adhesives                             0
Writing Instruments                  20
Files and Folders                    14
School Supplies                       0
Desk Accessories                      0
Notebooks                            10
Office Supplies                       0

9 rows selected.
```

3. Highest revenue-generating category.

```
SQL> SELECT * FROM (SELECT c.category_id, c.category_name, NVL(SUM(od.quantity * od.price), 0) AS total_revenue FROM category c LEFT JOIN product p
ON c.category_id = p.category_id LEFT JOIN order_details od ON p.product_id = od.product_id GROUP BY c.category_id, c.category_name ORDER BY total_r
evenue DESC) WHERE ROWNUM = 1;

CATEGOR CATEGORY_NAME             TOTAL_REVENUE
------- ------------------------- -------------
104     Art Supplies                   80000.00

1 row selected.
```

4. Average sales per category.

```
SQL> SELECT c.category_name, NVL(AVG(od.quantity * od.price), 0) AS average_sales FROM category c LEFT JOIN product p ON c.category_id = p.category_
id LEFT JOIN order_details od ON p.product_id = od.product_id GROUP BY c.category_name;

CATEGORY_NAME             AVERAGE_SALES
------------------------- -------------
Art Supplies                      80000
Paper Products                        0
Adhesives                             0
Writing Instruments               25000
Files and Folders                 21000
School Supplies                       0
Desk Accessories                      0
Notebooks                         20000
Office Supplies                       0

9 rows selected.
```

**7. Generate Business Analytics Reports**

**Report 1: Sales Performance Report**

- Total sales
- Number of orders
- Average order value
- Highest order value

```
SQL> SELECT COUNT(*) AS number_of_orders, SUM(total_amount) AS total_sales, AVG(total_amount) AS average_order_value, MAX(total_amount) AS highest_o
rder_value FROM orders;

NUMBER_OF_ORDERS   TOTAL_SALES AVERAGE_ORDER_VALUE HIGHEST_ORDER_VALUE
---------------- ------------- ------------------- -------------------
               5      13078.00             2615.60                5071

1 row selected.
```

**Report 2: Customer Analytics Report**

- Customer name
- Number of orders
- Total spending
- Purchase frequency

```
SQL> SELECT c.customer_name, COUNT(o.order_id) AS number_of_orders, NVL(SUM(o.total_amount), 0) AS total_spending FROM customer c LEFT JOIN orders o
 ON c.customer_id = o.customer_id GROUP BY c.customer_id, c.customer_name;

CUSTOMER_NAME             NUMBER_OF_ORDERS TOTAL_SPENDING
------------------------- ---------------- --------------
Rahul Raj                                1           4800
Priya Sharma                             1            307
Divya Devi                               1            400
Arun Kumar Raj                           1           2500
Ram                                      1           5071

5 rows selected.
```

**Report 3: Product Performance Report**

• Product name

• Quantity sold

• Revenue generated

```
SQL> SELECT p.product_name, NVL(SUM(od.quantity), 0) AS quantity_sold, NVL(SUM(od.quantity * od.price), 0) AS revenue_generated FROM product p LEFT
JOIN order_details od ON p.product_id = od.product_id GROUP BY p.product_id, p.product_name ORDER BY revenue_generated DESC;

PRODUCT_NAME              QUANTITY_SOLD REVENUE_GENERATED
------------------------- ------------- -----------------
Pencil Box                           20             80000
Pencil                               15             45000
Spiral Notebook                      14             21000
Gel Pen                              10             20000
Ball Pen                              5              5000
Notebook                              0                 0

6 rows selected.
```

**Report 4: Category Analysis Report**

• Category name

• Total products sold

• Total revenue

```
SQL> SELECT c.category_name, NVL(SUM(od.quantity), 0) AS total_products_sold, NVL(SUM(od.quantity * od.price), 0) AS total_revenue FROM category c L
EFT JOIN product p ON c.category_id = p.category_id LEFT JOIN order_details od ON p.product_id = od.product_id GROUP BY c.category_id, c.category_na
me ORDER BY total_revenue DESC;

CATEGORY_NAME             TOTAL_PRODUCTS_SOLD TOTAL_REVENUE
------------------------- ------------------- -------------
Art Supplies                               20      80000.00
Writing Instruments                        20      50000.00
Files and Folders                          14      21000.00
Notebooks                                  10      20000.00
Paper Products                              0           .00
Office Supplies                             0           .00
Adhesives                                   0           .00
Desk Accessories                            0           .00
School Supplies                             0           .00

9 rows selected.
```